

Join operations

First join operation:

1- Query definition:

This query retrieves employee information along with their attendance records by joining the EMPLOYEE and ATTENDANCE tables using the EMP_ID field.

2- Relational algebra:

π EMPLOYEE.EMP_ID, EMPLOYEE.FR_NAME,
EMPLOYEE.LA_NAME,

ATTENDANCE. ATT_ID, ATTENDANCE. ATT_DATE

(

EMPLOYEE $\bowtie$ EMPLOYEE.EMP_ID = ATTENDANCE.EMP_ID
ATTENDANCE

)

3- SQL statements and Sample of the output:

1	SELECT
2	EMPLOYEE.EMP_ID, EMPLOYEE.FR_NAME,EMPLOYEE.LA_NAME,
3	ATTENDANCE.ATT_ID,ATTENDANCE.ATT_DATE
4	FROM
5	EMPLOYEE
6	JOIN
7	ATTENDANCE ON EMPLOYEE.EMP_ID = ATTENDANCE.EMP_ID
8	
9	

Query result

Script output

DBMS output

Explain Plan

SQL history





Download

Execution time: 0.005 seconds

	EMP_ID	FR_NAME	LA_NAME	ATT_ID	ATT_DATE
1	301	Abdulaziz	Fahad	401	12/1/2025
2	301	Abdulaziz	Fahad	403	12/2/2025
3	302	Laila	Hussein	402	12/1/2025
4	303	Omar	Talal	404	12/2/2025
5	304	Nora	Abdullah	405	12/2/2025

Second join operation:

1- Query definition:

This query retrieves each employee's information along with their break times by joining the EMPLOYEE, ATTENDANCE, and BREAK tables using their related keys.

2- Relational algebra:

π EMPLOYEE.EMP_ID, EMPLOYEE.FR_NAME,
EMPLOYEE.LA_NAME,
BREAK.BREAK_START, BREAK.BREAK_END
(
EMPLOYEE $\bowtie$ EMPLOYEE.EMP_ID = ATTENDANCE.EMP_ID)
(ATTENDANCE
 $\bowtie$ ATTENDANCE.ATT_ID = BREAK.ATTENDANCE_ID BREAK
)

3- SQL statements and Sample of the output:

```
1  SELECT
2  | EMPLOYEE.EMP_ID, EMPLOYEE.FR_NAME, EMPLOYEE.LA_NAME,
3  | BREAK.BREAK_START, BREAK.BREAK_END
4  FROM
5  | EMPLOYEE
6  JOIN
7  | ATTENDANCE ON EMPLOYEE.EMP_ID = ATTENDANCE.EMP_ID
8  JOIN
9  | BREAK ON ATTENDANCE.ATT_ID = BREAK.ATTENDANCE_ID
```

Query result

Script output

DBMS output

Explain Plan

SQL history

  Download

Execution time: 0.008 seconds

	EMP_ID	FR_NAME	LA_NAME	BREAK_START	BREAK_END
1	301	Abdulaziz	Fahad	12:30 PM	01:00 PM
2	302	Laila	Hussein	01:00 PM	01:30 PM
3	303	Omar	Talal	12:00 PM	12:30 PM
4	304	Nora	Abdullah	10:00 AM	10:15 AM
5	304	Nora	Abdullah	02:00 PM	02:30 PM

Third join operation:

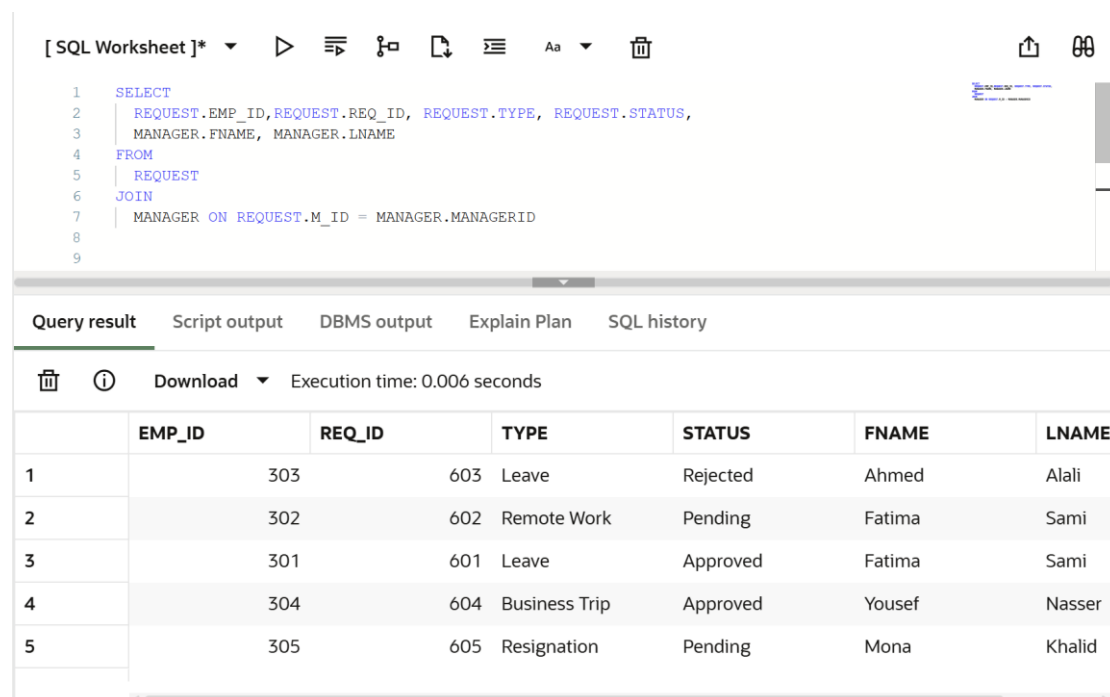
1- Query definition:

This query retrieves each employee's request details (type and status) along with the first and last name of the manager responsible for handling that request by joining the REQUEST and MANAGER tables using the manager ID.

2- Relational algebra:

$$\pi \text{ REQUEST.EMP_ID, REQUEST.REQ_ID, REQUEST.TYPE, REQUEST.STATUS, } \\ \text{MANAGER.FNAME, MANAGER.LNAME} \\ (\\ \text{REQUEST} \bowtie \text{REQUEST.M_ID} = \text{MANAGER.MANAGERID} \\ \text{MANAGER} \\)$$

3- SQL statements and Sample of the output:



The screenshot shows an SQL IDE interface. At the top, there's a toolbar with icons for running, saving, and other functions. Below the toolbar, the SQL query is entered in a text area. The query is a SELECT statement that joins the REQUEST and MANAGER tables on the condition REQUEST.M_ID = MANAGER.MANAGERID. The columns selected are REQUEST.EMP_ID, REQUEST.REQ_ID, REQUEST.TYPE, REQUEST.STATUS, MANAGER.FNAME, and MANAGER.LNAME. Below the query editor, there's a tabbed interface with 'Query result' selected. The query results are displayed in a table with 7 columns: EMP_ID, REQ_ID, TYPE, STATUS, FNAME, and LNAME. The table contains 5 rows of data.

```
[ SQL Worksheet ]*          
```

```
1 SELECT
2   REQUEST.EMP_ID,REQUEST.REQ_ID, REQUEST.TYPE, REQUEST.STATUS,
3   MANAGER.FNAME, MANAGER.LNAME
4 FROM
5   REQUEST
6 JOIN
7   MANAGER ON REQUEST.M_ID = MANAGER.MANAGERID
8
9
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.006 seconds

	EMP_ID	REQ_ID	TYPE	STATUS	FNAME	LNAME
1	303	603	Leave	Rejected	Ahmed	Alali
2	302	602	Remote Work	Pending	Fatima	Sami
3	301	601	Leave	Approved	Fatima	Sami
4	304	604	Business Trip	Approved	Yousef	Nasser
5	305	605	Resignation	Pending	Mona	Khalid

Fourth join operation:

1- Query definition:

This query retrieves only the employee requests that have been approved. It joins the REQUEST and EMPLOYEE tables using the employee ID and displays the approved request details along with the employee's first and last name.

2- Relational algebra:

$$\pi \text{ EMPLOYEE.EMP_ID, EMPLOYEE.FR_NAME, EMPLOYEE.LA_NAME, REQUEST.REQ_ID, REQUEST.TYPE, REQUEST.STATUS } (\sigma \text{ REQUEST.STATUS = 'Approved' } (\text{REQUEST} \bowtie \text{REQUEST.EMP_ID = EMPLOYEE.EMP_ID EMPLOYEE}))$$

3- SQL statements and Sample of the output:

```
1 SELECT
2   EMPLOYEE.EMP_ID,EMPLOYEE.FR_NAME, EMPLOYEE.LA_NAME,
3   REQUEST.REQ_ID,REQUEST.TYPE,REQUEST.STATUS
4 FROM REQUEST
5 JOIN EMPLOYEE
6   ON REQUEST.EMP_ID = EMPLOYEE.EMP_ID
7 WHERE REQUEST.STATUS = 'Approved';
8
9
```

Query result						
Script output						
DBMS output						
Explain Plan						
SQL history						
Download Execution time: 0.007 seconds						
	EMP_ID	FR_NAME	LA_NAME	REQ_ID	TYPE	STATUS
1	301	Abdulaziz	Fahad	601	Leave	Approved
2	304	Nora	Abdullah	604	Business Trip	Approved

Fifth join operation:

1- Query definition:

This query retrieves only the employee requests that have been rejected. It joins the REQUEST and EMPLOYEE tables using the employee ID and displays the employee's first and last name along with the request ID, type, and rejection status.

2- Relational algebra:

$$\pi \text{ EMPLOYEE.EMP_ID, EMPLOYEE.FR_NAME, EMPLOYEE.LA_NAME, REQUEST.REQ_ID, REQUEST.TYPE, REQUEST.STATUS } (\sigma \text{ REQUEST.STATUS = 'Rejected' } (\text{REQUEST} \bowtie \text{REQUEST.EMP_ID = EMPLOYEE.EMP_ID EMPLOYEE}))$$

3- SQL statements and Sample of the output:

```
1 SELECT
2   EMPLOYEE.EMP_ID,EMPLOYEE.FR_NAME, EMPLOYEE.LA_NAME,
3   REQUEST.REQ_ID,REQUEST.TYPE,REQUEST.STATUS
4 FROM REQUEST
5 JOIN EMPLOYEE
6   ON REQUEST.EMP_ID = EMPLOYEE.EMP_ID
7 WHERE REQUEST.STATUS = 'Rejected';
8
9
```

	EMP_ID	FR_NAME	LA_NAME	REQ_ID	TYPE	STATUS
1	303	Omar	Talal	603	Leave	Rejected

Aggregate functions

First aggregate function (COUNT):

1- Query definition:

This query calculates the total number of employees in the EMPLOYEE table using the COUNT aggregate function.

2- Relational algebra:

$\mathcal{F}$ COUNT EMPLOYEE.EMP_ID (EMPLOYEE)

3- SQL statements and Sample of the output:

```
1  SELECT COUNT(*) AS total_employees
2  FROM EMPLOYEE;
3
4
5
6
7
8
9
```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.005 seconds

	TOTAL_EMPLOYEES
1	5

Second aggregate function (MIN):

1- Query definition:

This query retrieves the earliest registered employee by selecting the smallest employee ID using the MIN aggregate function.

2- Relational algebra:

$\mathcal{F}$ MIN EMP_ID (EMPLOYEE)

3- SQL statements and Sample of the output:

```
1  SELECT MIN(EMP_ID) AS lowest_employee_id
2  FROM EMPLOYEE;
3
4
5
6
7
8
9
```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.003 seconds

	LOWEST_EMPLOYEE_ID
1	301

Third aggregate function (MAX):

1- Query definition:

This query retrieves the most recently registered employee by selecting the highest employee ID using the MAX aggregate function.

2- Relational algebra:

$\mathcal{F}$ MAX EMP_ID (EMPLOYEE)

3- SQL statements and Sample of the output:

```
1  SELECT MAX(EMP_ID) AS newest_employee_id
2  FROM EMPLOYEE;
3
4
5
6
7
8
9
```

Query result		Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.003 seconds					
		NEWEST_EMPLOYEE			
1		305			